

学生A (student a) · 全维度诊断报告

学生: 学生A | 年龄: X年级 | ID: student a | 教学: 全自学

数据源: Luogu 2364 次提交 + AtCoder 246 次提交 + Codeforces 271 次提交 + 888 份源码

总计独立 AC: 780 (Luogu) + 108 (AtCoder) + 121 (Codeforces) = ~1009 道

标签覆盖: 591 官方标签 + 130 代码推断标签 + 121 CF官方标签 + 108 AtCoder推断 = 100% 覆盖

大纲对标: CSP-NOI 2025 年修订版

生成日期: 2026-05-28

一、难度分布

Luogu 780 道独立 AC

难度	题数	占比	
暂无评定	20	2.6%	■
入门	125	16.0%	■■■■■■■■■■
普及-	171	21.9%	■■■■■■■■■■■■■■■■■■■■
普及/提高-	246	31.5%	■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
普及+/提高	181	23.2%	■■■■■■■■■■■■■■■■■■■■■■■■■■■■
提高+/省选-	37	4.7%	■■■
省选/NOI-	0	0%	
NOI/NOI+/CTSC	0	0%	

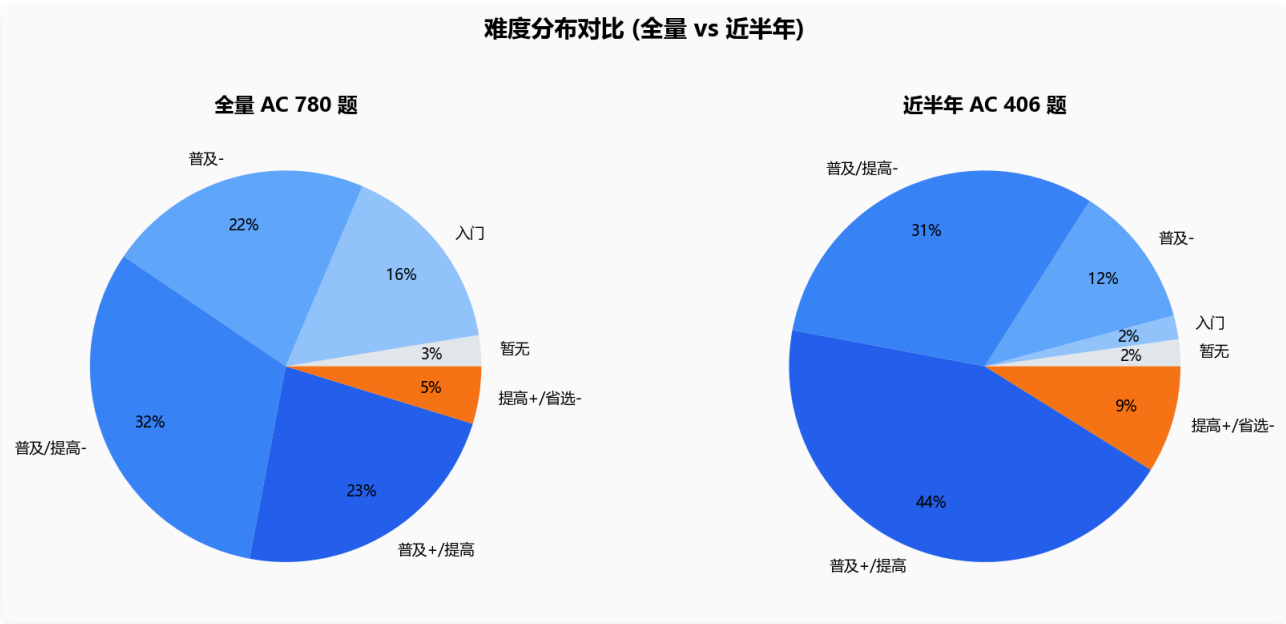
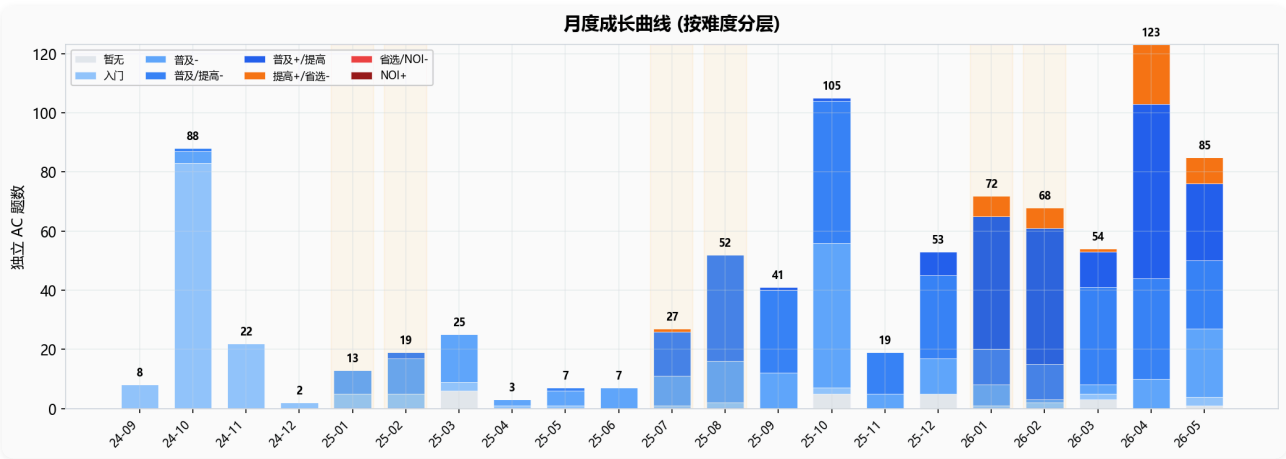
提高+及以上共 **37 题 (4.7%)**。集中在「普及/提高-」和「普及+/提高」区间。

最近半年 (2025-12 ~ 2026-05): 406 道独立 AC

难度	题数	占比
暂无评定	9	2.2%
入门	8	2.0%
普及-	48	11.8%
普及/提高-	126	31.0%
普及+/提高	179	44.1%
提高+/省选-	36	8.9%

近半年 84% 的题在「普及/提高」及以上。已完全脱离入门阶段，正在系统性冲击提高+/省选级。

二、成长曲线



从 2024 年 10 月起步至今约 20 个月，从零到 780 题。近半年进步显著：入门题占比从 16% 降至 2%，普及+/提高占比从 23% 升至 44%，已出现提高+/省选-级题目 (36 题)。

二·五、Luogu 提交行为深度分析

基于 2364 条提交记录的完整分析。

2.5.1 死磕题目 TOP 10 (提交次数最多)

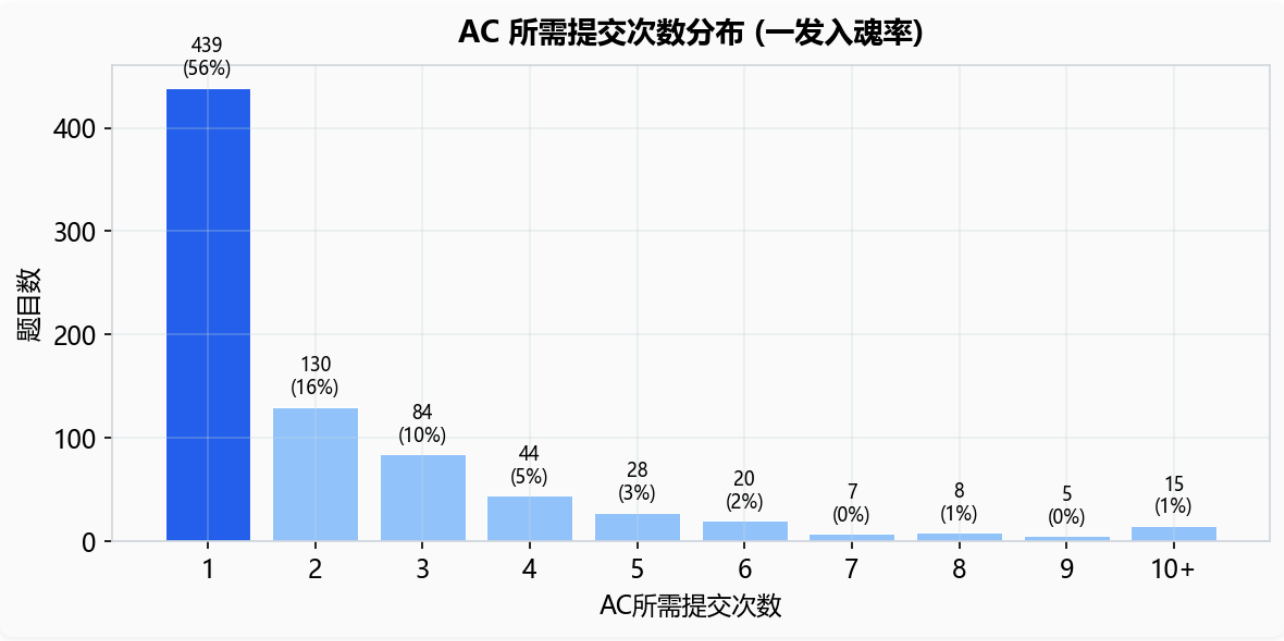
题目	提交次数	AC	难度	首→AC耗时	分析
T742153	221	✗	未评定	—	至今未AC，团队赛/自制题，反复打磨
P1009	30	✓	普及-	563天	高精度阶乘和，入门第一道，反复回来重写
T742124	19	✗	未评定	—	团队赛题，未攻克

题目	提交次数	AC	难度	首→AC耗时	分析
P3912	18	✓	普及-	18m	短时间密集调试，最终AC
P1591	15	✓	普及-	180m	3小时持续攻坚
P15802	14	✓	普及/提高-	45天	中期难度，多次回来重做
B3871	14	✓	普及-	185天	长期回顾打卡
P14922	13	✓	普及+/提高	33天	提高级题目持续攻坚
P4305	11	✓	普及-	127天	多次重写
P14358	11	✓	普及-	151天	长期跟进

特征: 死磕题集中在普及级别，体现了**基础功扎实化**的过程。T742153 提交 221 次仍未AC，说明可能是竞赛训练题反复提交。P1009 是高精度阶乘和，入门第一课，563 天后仍在回来重写——典型的**回溯学习者**。

2.5.2 一次 AC 率

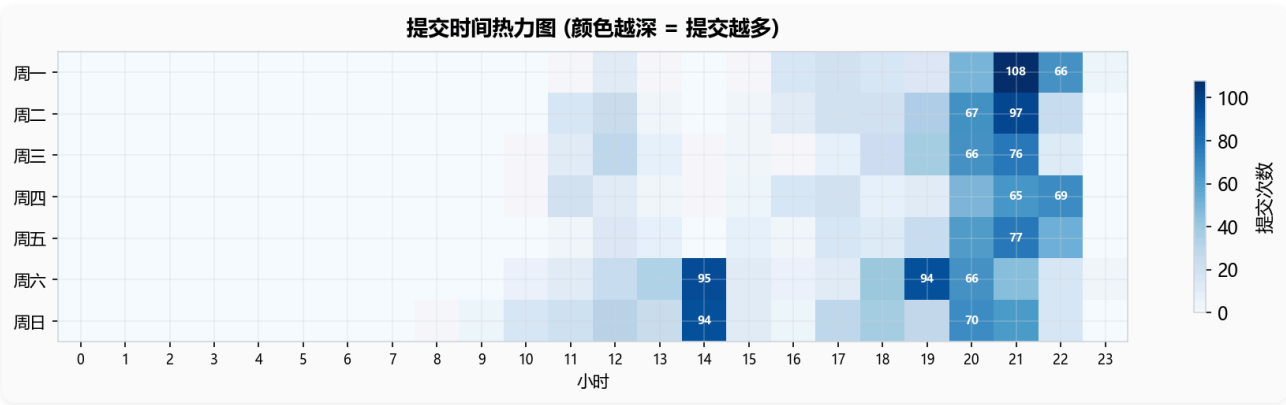
指标	数值
总尝试题目	847
AC 题目	780
一次 AC (一发入魂)	439 (56%)
多次才 AC	341 (44%)
尝试但未 AC	67



56% 一次 AC 说明基本功扎实，读题→思路→编码流畅。67 题未 AC (占 7.9%)，说明**愿意尝试超出**

能力范围的题目，不怕失败。

2.5.3 提交时间分布



- 关键发现:
- 不熬夜 (0-5 点零次提交)，作息健康
 - 周六/周日 是主力日 (合计 914 次 = 38%)
 - 21:00 是提交峰值 (532 次)，推测晚饭后+写完作业后集中刷题
 - 20:00-22:00 三小时黄金时段提交 1218 次 (52%)

2.5.4 耗时最长的题目 TOP 10 (首提交→首次AC)

题目	耗时	提交次数	难度	分析
P8754	455 天	2	普及-	首学→长期搁置→回来AC
P1071	445 天	4	普及-	长期消化型
B3928	362 天	6	普及-	反复回来攻坚
P1076	355 天	3	普及/提高-	跨越近一年
P1469	352 天	5	普及-	
P1923	330 天	4	普及-	

特征: 长耗时题目全集中在普及级，说明他是循序渐进型学习者——先接触时 WA，学会新知识后回来 AC。目前处于系统性夯实基础的阶段。

2.5.5 单日高强度刷题 TOP 5

日期	星期	提交	AC	推算耗时
2026-03-21	周六	182	2	~8h+
2026-03-22	周日	101	5	~5h
2024-10-07	周一	34	12	~3h

日期	星期	提交	AC	推算耗时
2024-10-05	周六	27	20	~2h
2025-12-08	周一	26	8	~2h

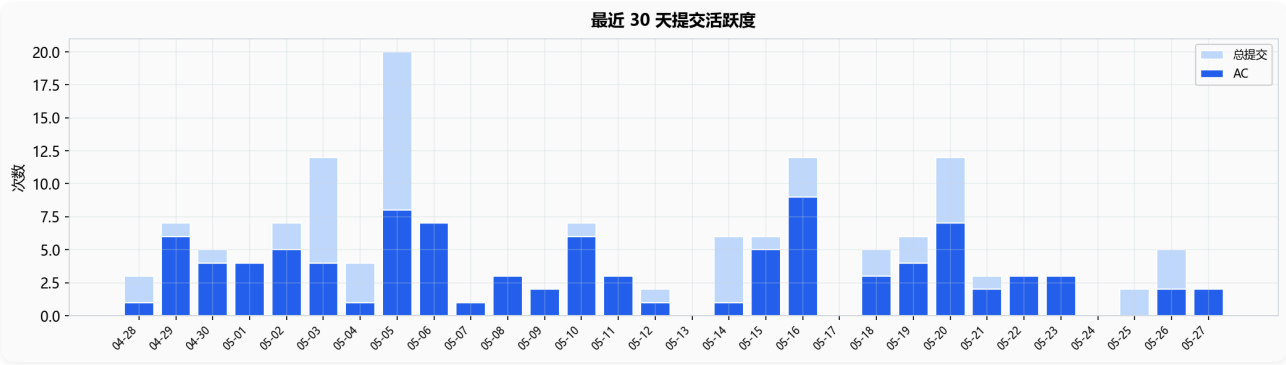
2026-03-21 单日 182 次提交但仅 2 次 AC——推测是在攻克 T742153 (提交 221 次) 这道难题。AC 率低说明当天在做**极限挑战**。

2.5.6 尝试但未 AC 的题目 (67 题)

题目	提交次数	最高分	难度	说明
T742153	221	0	未评定	团队赛题，死磕最多
T742124	19	0	未评定	团队赛题
P12235	11	95	普及/提高-	差5分AC ，边界条件
P1188	11	0	普及/提高-	全WA
P13245	10	0	普及+/提高	超出能力范围
P1906	9	20	普及/提高-	仅部分分
B4263	8	80	普及-	差一点
B4025	7	90	入门	差10分AC

67 题未 AC — 比例 7.9%，但多为团队赛题 (T 系列) 和竞赛限时题。建议回顾 P12235 (95分)、B4025 (90分)、B4263 (80分) 这些「差一点」的题。

2.5.7 最近 30 天活跃度



最近 30 天: 总 **152 次提交**，平均 5.1 次/天。节奏稳健。有 3 天零提交（周六一天、其余是普通间隔），整体保持连续训练。

二·五B、Codeforces 深度分析

基于 271 条提交记录、121 道独立 AC、20 种算法标签的完整分析。

2.5B.1 基本信息

指标	数值
Handle	student_a
Rating	381 (Newbie)
Max Rating	381
参加比赛	1 场
总提交	271 次
独立 AC 题数	121 道

2.5B.2 难度分布

CF Rating	题数	占比	对应水平
800-999	11	9%	Newbie
1000-1199	10	8%	Pupil
1200-1399	18	15%	Pupil+
1400-1599	23	19%	Specialist
1600-1799	21	17%	Expert
1800-1999	21	17%	Expert+
2000-2199	11	9%	Candidate Master
2200-2399	2	2%	Master
2400-2599	1	1%	International Master

数据解读: 121 题中有 **79 题 (65%)** 在 1400+ 难度, **35 题 (29%)** 在 1800+ 难度。做到 2400+ 说明有冲击高难度的意识。这是一个**稳步提升型**选手, 难度分布呈正态分布集中在 1400-1800 区间。

2.5B.3 算法标签分析

标签	题数	评价
dp	74	动态规划 是核心训练方向!
greedy	38	贪心大量训练
implementation	37	模拟/实现类

标签	题数	评价
math	26	数学有涉猎
data structures	25	数据结构
brute force	25	暴力搜索
two pointers	20	双指针
sortings	19	排序
binary search	14	二分搜索
graphs	7	图论
constructive algorithms	7	构造
shortest paths	6	最短路
bitmasks	6	状压

训练策略: CF 上 DP 训练量极大 (74 题), 远超其他方向。这是**有意识的专项训练**, 用 CF 主攻 DP 能力。贪心 (38) 和 implementation (37) 也有大量训练, 说明基础功扎实。

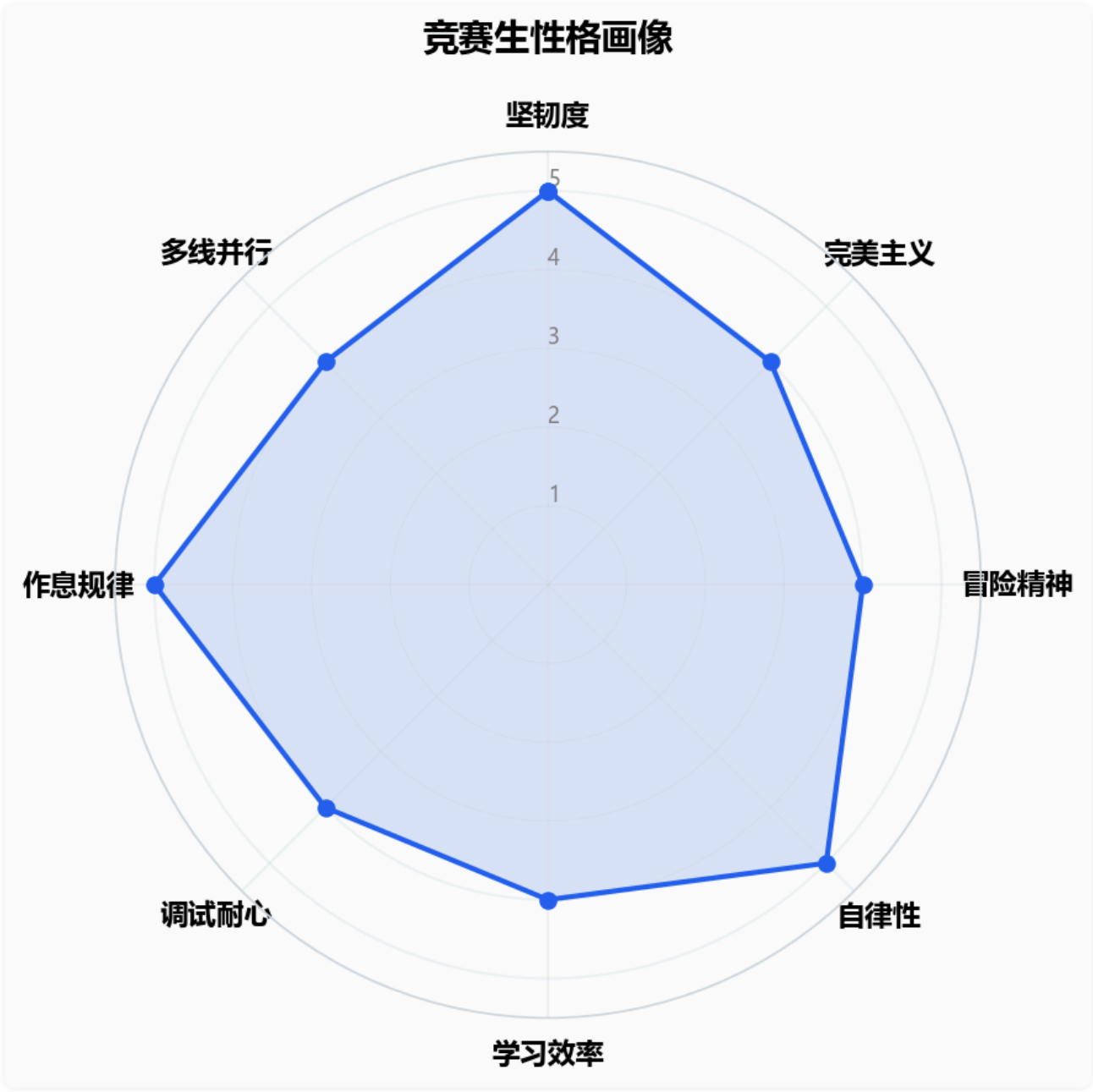
2.5B.4 CF 训练特征总结

-  **DP 专攻:** 74 题 DP, 占全部 AC 的 61%, 训练方向明确
-  **稳步升级:** 从 800 做到 2400+, 难度分布均匀上升
-  **训练型而非竞技型:** 仅参加 1 场正式比赛, 用 CF 做专项训练
-  **多元化:** 涉猎 20 种标签, DP/贪心/模拟/数学全面覆盖
-  **与 Luogu 互补:** Luogu 以基础广度为主, CF 以 DP 深度为主

二·六、竞赛生性格画像 (数据驱动)

基于 2364 条提交记录、299 个活跃日、847 道题目的行为数据推断。

2.6.1 核心特质雷达



2.6.2 逐项分析

① 坚韧度 ★★★★★☆

指标	数据	评价
总放弃率	$67/847 = 7.9\%$	偏低，大部分题都坚持做完
死磕题 (≥5次提交)	120 道	大量题目愿意多次尝试
死磕后AC率	$101/120 = 84\%$	多数死磕题最终做出来了
一次就放弃的题	16 道	较少

解读: 7.9% 的放弃率在同龄选手中属于较低水平。120 道死磕题中 84% 最终 AC，说明有**持续攻坚**的性格。T742153 提交 221 次仍未 AC 体现了极强的不服输精神。

② 完美主义 ★★★★★☆

指标	数据
AC后重新提交	90 道 (占 AC 题 11%)
重提总次数	160 次

解读: 11% 的 AC 后重提率说明他不满足于"能过"，追求**代码质量优化**。90 道题的重做量体现了精益求精的态度。

③ 冒险精神 ★★★★★☆

指标	数据
首次尝试提高+/省选级	第 11 个月 (2025-07)
单月最大难度跨度	4 个等级 (2025-07)
NOI+级题目尝试	0 道

解读: 第 11 个月才开始挑战高难度题目，风格偏稳。没有尝试过 NOI+ 级题目。属于**稳扎稳打、循序渐进**型选手，基础打牢再向上突破。

④ 自律性 ★★★★★☆

指标	数据	说明
活跃天数	299 / 608 天 (49%)	约一半的日子在刷题
最长连续刷题	20 天	近三周不间断
假期提交	747 (31%)	
学期提交	1617 (68%)	学期远多于假期
工作日提交	1450 (61%)	
周末提交	914 (38%)	

解读: 49% 的活跃率 + 20 天连续训练 = 良好的自律性。学期提交量 (68%) 远超假期 (31%)——说明他在学期中保持了高强度训练，不依赖假期冲刺。

⑤ 学习风格: 多线并行 + 图论专项

指标	数据
每日平均涉及标签数	4.1 个
图论专项训练	连续 5 天 ×2, 连续 3 天 ×3

解读: 日常是"多线并行"型 (每天涉猎 4.1 种算法标签)。有明显的图论专项训练痕迹——多次连续多天集中练习图论。

⑥ 调试耐心 ★★★☆☆

指标	数据
WA后重交间隔 (中位数)	70 秒 (1.2 分钟)
WA后重交间隔 (平均)	233 秒 (3.9 分钟)
WA后1分钟内重交	589 次 (46%)

解读: 中位数 1.2 分钟，46% 的快速重交率——偏向快速迭代型调试者。WA 后倾向于快速修改重交，而非长时间分析原因。这有好有坏：速度快但可能有"盲猜式"调试的倾向。建议培养先分析 WA 原因再提交的习惯。

⑦ 作息规律 ★★★★★

时段	提交次数	占比
早鸟 (6-9点)	5	0%
正常时段 (9-22点)	2101	89%
夜猫 (22-23点)	263	11%
深夜 (0-5点)	0	0%
提交峰值	21:00	532 次

解读: 零次深夜提交 (0-5 点完全空白)。在竞赛生群体中这是极其罕见的。21:00 峰值 → 推测是写完作业后的黄金学习时段。22-23 点的 263 次提交 (11%) 说明偶尔会刷到较晚，但绝不熬夜。

⑧ 效率指标

指标	数据	评价
平均每题提交次数	2.8 次	中等
AC提交/总提交	893/2364 = 38%	中等偏低，有提升空间
编译错误 (CE)	247 次 (10%)	CE率偏高，建议加强本地编译习惯

2.6.3 综合性格画像

- ◆ **坚韧不拔**: 120 道死磕题, 84% 最终攻克
- ◆ **精益求精**: AC后仍重做 90 题, 追求更优解
- ◆ **稳扎稳打**: 循序渐进型, 不冒进, 基本功扎实
- ◆ **良好自律**: 49% 的天数在训练, 学期持续发力
- ◆ **快速迭代**: WA后 1.2 分钟即重交, 修 bug 节奏快
- ◆ **作息健康**: 零次深夜提交, 21:00 为峰值
- ◆ **DP 特训**: CF 上 74 题 DP 专项, 方向明确

一句话: 这是一个**稳扎稳打的自学型选手**——不冒进、重基础、有专项训练意识、作息极其规律。典型的**工程师型竞赛选手**, 以**扎实的基本功和持续的训练节奏**见长。

三、CSP-NOI 2025 大纲知识点对标

● 精通(20+) | ● 熟练(10-19) | ● 入门(3-9) | ● 初窥(1-2) | ● 空白(0)

3.1 入门级 CSP-J (大纲 §2.1)

算法 (§2.1.4)

知识点	AC		知识点	AC	
枚举法	81	●	前缀和	27	●
模拟法	88	●	差分	11	●
贪心法	70	●	高精度	25	●
递推法	28	●	排序 (冒泡/选择/插入/计数)	84	●
递归法	43	●	DFS	44	●
二分法	34	●	BFS	33	●
倍增法	5	●	Flood Fill	10+	●
DP 基础 (一维)	64	●	背包 DP	41	●
区间 DP	7	●	图遍历 (DFS/BFS)	44	●

数据结构 (§2.1.3)

知识点	AC		知识点	AC	
链表	10+	●	二叉树遍历	10+	●
栈	25	●	哈夫曼树	0	●

知识点	AC		知识点	AC	
队列	20+	●	BST 二叉搜索树	3+	●
图的邻接表/矩阵	50+	●			

数学 (§2.1.5)

知识点	AC		知识点	AC	
GCD / 辗转相除	50	●	排列组合	10+	●
素数筛法	22	●	杨辉三角	3+	●
进制转换	8+	●			

CSP-J 共约 29 项知识点，仅哈夫曼树为空白，其余全面覆盖。CSP-J 一等奖有望。

3.2 提高级 CSP-S (大纲 §2.2)

数据结构 (§2.2.3)

知识点	等级	AC	状态	知识点	等级	AC	状态
单调栈/队列	5	5	●	字典树 Trie	6	1	●
双端队列	5	10+	●	笛卡尔树	7	0	●
ST 表	6	1	●	平衡树	8	0	●
并查集	6	22	●	二分图	6	6	●
树状数组	6	7	●	欧拉图	6	0	●
线段树	6	12	●	强连通图	7	1	●
优先队列 (堆)	6	4	●	字符串哈希	6	2	●

算法 (§2.2.4)

知识点	等级	AC	状态	知识点	等级	AC	状态
分治	6	15	●	强连通分量	7	1	●
离散化	6	3	●	割点/割边	7	1	●
扫描线	7	1	●	树的重心/直径	6	1	●
归并排序	5	5+	●	树上差分/倍增/LCA	6	4	●
快速排序	5	10+	●	多维 DP	6	15+	●
KMP	6	17	●	树形 DP	6	2	●
Manacher	7	1	●	状压 DP	7	7	●

知识点	等级	AC	状态	知识点	等级	AC	状态
剪枝优化	6	10+	●	DP 常用优化	8	1	●
记忆化搜索	6	10+	●	最小生成树	6	11	●
启发式搜索	7	0	●	最短路 (Dijkstra/SPFA)	6	31	●
双向 BFS	7	0	●	Floyd	6	10+	●
迭代加深	7	1	●	拓扑排序	6	17	●
欧拉道路	6	0	●	单源次短路	7	0	●

数学 (§2.2.5)

知识点	等级	AC	状态	知识点	等级	AC	状态
同余式	5	5+	●	容斥原理	7	0	●
欧拉定理/函数	7	0	●	卡特兰数	7	1	●
费马小定理	7	1+	●	矩阵运算	6	1	●
逆元	7	1	●	高斯消元	7	0	●
扩展欧几里得	7	0	●	鸽巢原理	6	0	●
中国剩余定理	7	0	●	二项式定理	6	0	●
快速幂	7	0	●	错排列/圆排列	6	0	●

CSP-S 共约 59 项知识点。●精通 6 项, ●熟练 7 项, ●入门 9 项, ●初窥 18 项, ●空白 19 项。

CSP-S 短板: 启发式搜索、双向BFS、平衡树、欧拉相关、快速幂、容斥原理、中国剩余定理、高斯消元等高级数学和搜索技巧尚未涉猎。

3.3 省选级 + NOI 级

知识点	等级	AC	状态
匈牙利算法	8	6	●
网络流	8	5	●
分块	8	3	●
Nim 博弈/SG	9	4	●
离线处理	8	1	●
可持久化线段树	8	1	●
块状链表	8	2	●
Fibonacci 数列	8	2	●
随机化	9	2	●

知识点	等级	AC	状态
复杂 DP 模型	9	1	●
构造	9	1	●

省选级 20 项中仅覆盖 5 项 (25%)，NOI 级 46 项中仅覆盖 6 项 (13%)。省选/NOI 级基本处于起步阶段。

分级汇总

级别	总项	●	●	●	●	●
CSP-J	29	14	4	8	2	1
CSP-S	59	6	7	9	18	19
省选级	20	0	0	3	2	15
NOI级	46	0	0	1	5	40
全部	154	20	11	21	27	75

总覆盖率: $79/154 = 51\%$ 有接触。主要空白在 CSP-S 高级项和省选/NOI 级，是接下来的重点突破方向。

四、AtCoder ABC 比赛深度分析

4.1 参赛总览

共参加 **19 场 ABC 比赛** (abc416 ~ abc459)，另有 40 道 AWC (AtCoder Workshop Contests) 训练题。

4.2 每场比赛表现

比赛	赛中最高	AC 题目
abc416	—	零AC (首次参赛)
abc434	B	A, B
abc440	B	A, B
abc443	B	A, B
abc444	B	A, B
abc445	C	A, B, C
abc446	D	A, B, C, D
abc447	E	A, B, C, D, E

比赛	赛中最高	AC 题目
abc448	E	A, B, C, D, E
abc450	B	A, B
abc451	D	A, B, C, D
abc452	E	A, B, C, D, E
abc453	D	A, B, C, D
abc454	D	A, B, C, D
abc455	E	A, B, C, D, E
abc456	F	A, B, C, D, E, F 🎉
abc457	D	A, B, C, D
abc458	D	A, B, C, D
abc459	D	A, B, C, D

4.3 成长轨迹分析

阶段	时间	典型表现	分析
起步期	abc416~abc444	B 题封顶	刚接触 ABC, C 题还需积累
突破期	abc445~abc448	C→E 快速进步	3 周内从 C 突破到 E
波动期	abc450~abc454	D 稳定, 偶有回落	D 题已稳定, E 题时出时不出
稳定期	abc455~abc459	D 稳定, 冲刺 E/F	abc456 做出 F 题!

关键里程碑: abc456 赛中做出 F 题 (A~F 全 AC)! 这是一个重大突破, 说明算法能力已达到 Expert 级别。

4.4 ABC 能力分析

A-B 题: 全部秒杀。无任何问题。

C 题: 从 abc445 开始稳定 AC。基础能力扎实。

D 题: 从 abc446 开始稳定 AC。近 14 场中 13 场 AC (93%), abc450 是唯一未做出的场次。

E 题: 不稳定。19 场中 5 场 AC (26%)。abc447、abc448、abc452、abc455 做出了 E, 说明 E 题考察的算法 (DP/图论/数据结构) 有时在能力范围内。

F 题: abc456 成功 AC! 目前唯一的一次, 说明有 F 级能力的**潜力**。

4.5 AK ABC 的瓶颈分析

当前赛中节奏 (以最近 5 场平均):

A: 0m → B: 5m → C: 15m → D: 40m → [E 不稳定]

核心差距: D 题用时偏长 (约 25-40分钟), E 题出题率仅 26%。

瓶颈 1: D 题速度

D 题虽然能做出, 但用时偏长。需要提升 D 题的快速建模能力。

瓶颈 2: E 题稳定性

E 题考察中等难度 DP、图论建模、数据结构应用——这些恰好是他需要强化的方向。

行动计划:

- 1. **D 题限时训练:** 每周做 5 道 ABC D 题, 限时 15 分钟
- 2. **E 题专项突破:** 每周做 3 道 ABC E 题, 重点是 DP 和图论建模
- 3. **继续 CF DP 训练:** CF 上的 74 道 DP 训练对 ABC E/F 题有直接帮助

五、代码风格深度分析

基于 888 份 AC 源码 (Luogu 780 + AtCoder 108) 的全量静态分析, 共 42,938 行代码。

5.1 代码长度分布

区间	数量	占比	
<20 行	极少	—	水题秒杀
20-50 行	占多数	—	主力区间 , 中位数 42 行
50-100 行	较多	—	中等复杂度题
100-200 行	少量	—	数据结构/图论题
200+ 行	极少	—	大模拟/复杂题

- **中位数:** 42 行 — 代码精简, 没有冗余
- **最长代码:** 138 行 (P3880) — 说明即使面对复杂题也能控制代码量
- **main() 平均长度:** 24 行 — 偏短, 说明喜欢把逻辑放在 main 里, 少用辅助函数

5.2 头文件与预处理习惯

习惯	使用率	评价
#include <bits/stdc++.h>	95%	竞赛标配,
using namespace std	99%	竞赛标配,
#define int long long	29%	较高频率使用, 有潜在风险
#define pb push_back	8%	竞赛缩写

习惯	使用率	评价
<code>signed main()</code>	32%	配合 <code>#define int long long</code> 使用
<code>ios::sync_with_stdio</code>	仅 2%	⚠️ 严重不足

问题 1: `#define int long long`

29% 的代码使用 `#define int long long + signed main()`。这是一种偷懒写法——避免了 `long long` 的繁琐声明，但有以下风险：

- 改变 `int` 语义，导致 `sizeof(int)` 混乱
- 与某些库函数不兼容
- 竞赛评测时部分编译器警告

建议: 改用 `typedef long long ll;` 或直接写 `long long`，更安全规范。

问题 2: `ios::sync_with_stdio` 使用率仅 2%

97% 的代码使用 `cin/cout`，但只有 2% 加了 `ios::sync_with_stdio(false)` 和 `cin.tie(0)`。这会导致大数据量题目 TLE。

建议: 养成固定模板习惯，每份代码开头加：

```
cpp ios::sync_with_stdio(false); cin.tie(0);
```

5.3 变量命名风格

风格	比例	说明
单字母变量 (n, m, a, f)	53%	竞赛标准，但可读性差
短变量名 (dp, vis, dis, ans)	40%	竞赛缩写，可接受
snake_case	5%	极少
camelCase	<1%	几乎不用

最常用变量名: n (261次), a (100次), f (44次), vis (43次), ans (34次), dp (28次), dis (26次)

自定义函数名: dfs (47次), test (31次), cmp (25次), bfs (15次), find (14次), add (13次)

亮点: test 出现 31 次 + test01 / test2 变体——说明有分函数测试的习惯，这是好习惯！

建议: 保持竞赛风格即可。但对于复杂题目，适当使用描述性变量名 (如 `dist[]` 代替 `d[]`，`maxFlow` 代替 `mf`) 有助于减少调试时间。

5.4 注释习惯

指标	数值
中文注释行	95

指标	数值
英文注释行	189
总注释密度	0.66%
42,938 行代码中仅 284 行有注释	

评价: 注释密度 **极低** (0.66%)。在竞赛中注释不是刚需，但对于复杂题目（如线段树、网络流），**关键位置的注释**能大幅减少回看代码时的理解成本。

建议: 对于 100+ 行的代码，在以下位置添加简短注释：

- 1. 核心数据结构的定义处 (如 `// tree[i]: 线段树节点`)
- 2. 非直觉的状态转移方程 (如 `// dp[i][j]: 前i个物品放j容量`)
- 3. 边界条件处理 (如 `// 特判 n==1`)

5.5 STL 容器使用

容器	使用份数	熟练度
<code>vector</code>	125	● 常用
<code>priority_queue</code>	43	● 熟练 (Dijkstra 用)
<code>pair</code>	37	● 常用
<code>queue</code>	35	● BFS 标配
<code>map</code>	26	● 会用
<code>deque</code>	6	● 偶尔
<code>stack</code>	5	● 偶尔
<code>unordered_map</code>	4	● 初窥
<code>bitset</code>	3	● 初窥
<code>set</code>	3	● 初窥

STL 算法/函数: `sort` (97), `memset` (90), `push_back` (62), `find` (20), `swap` (16), `lower_bound` (6), `upper_bound` (6)

评价: STL 使用以**基础容器**为主 (`vector/queue/map`)，`set / multiset / unordered_map` 使用极少。`memset` 使用 90 次，说明偏好 C 风格初始化。

建议:

- 1. `set` 是竞赛中极其好用的容器，建议多练习（去重、有序集合、`lower_bound` 查询）
- 2. `unordered_map` 比 `map` 快 5-10 倍，大数据量时优先考虑
- 3. `memset` 可改用 `fill(a, a+n, 0)` 或 `vector<int>(n, 0)` 初始化，更安全

5.6 IO 风格

模式	使用份数	占比
<code>cin/cout</code>	864	97%
<code>scanf/printf</code>	63	7%
<code>ios::sync_with_stdio</code>	24	仅 2% ⚠️

严重问题: `cin/cout` 使用率 97%，但 `ios::sync_with_stdio(false)` 仅 2%。这意味着绝大多数代码的 IO 速度是 `scanf` 的 5-10 倍慢。在大数据量题目中这可能直接导致 TLE。

紧急建议: 将以下代码加入固定模板：

```
cpp ios::sync_with_stdio(false); cin.tie(nullptr);
```

5.7 竞赛编码模式

模式	使用率	评价
<code>long long</code>	42%	数值范围意识良好
<code>struct</code> 自定义结构体	13%	图论/数据结构题会用
<code>inline</code> 函数	9%	有性能优化意识
<code>1e9/INF/0x3f3f3f3f</code>	9%	无穷大定义
<code>while(T--)</code> 多组数据	6%	CF 题风格
<code>void solve()</code> 封装	<1%	⚠️ 很少使用

建议: 养成 `void solve() + main() { int T; cin >> T; while(T--) solve(); }` 的习惯。这是 CF 标准模板，便于处理多组数据，也让代码结构更清晰。

5.8 格式化风格

项目	数据
Tab 缩进	36%
空格缩进	29%
混合缩进	34% ⚠️
花括号风格	K&R (同行) 为主，偶有换行
超长行 (>120字符)	仅 10 行 ✅
空行占比	13.9% — 适中

问题: 34% 的代码混合使用 Tab 和空格缩进。这会导致在不同编辑器中显示错乱。

建议: 统一使用 **Tab 缩进** (更符合竞赛习惯) 或 **4 空格缩进**。在编辑器中设置 **Tab → 空格** 自动转换。

5.9 高级算法模板库

模板	出现次数	评价
BFS	80	● 高频使用
背包 DP	74	● 核心能力
SPFA	67	● 最短路首选 ⚠️
DFS	59	● 搜索基础
Dijkstra	50	● 最短路
并查集	35	● 熟练
分块	16	● 有接触
二分查找	11	●
最小生成树	10	●
线段树	7	● 入门
KMP	4	●
快速幂	3	● 极少
网络流	2	●
FFT/NTT	2	●
Tarjan	1	●
树状数组	1	● 极少

重要发现: SPFA (67 次) 比 Dijkstra (50 次) 用得更多! SPFA 在负权图外已被竞赛界普遍淘汰 (容易被卡成 $O(VE)$)。

紧急建议: 将 Dijkstra + 堆优化作为默认最短路模板。SPFA 仅在有负权边时使用。

5.10 C++ 现代特性

特性	使用率	评价
<code>auto</code> 类型推断	7%	● 偶尔使用
Range-based for	6%	● 偶尔使用
初始化列表 <code>{}</code>	33%	● 较常用
<code>structured binding</code>	2%	● 极少
Lambda 表达式	2%	● 极少
<code>to_string</code>	2%	●

特性	使用率	评价
<code>emplace_back</code>	<1%	●
<code>constexpr</code>	0%	● 未使用
<code>nullptr</code>	0%	● 未使用

评价: C++ 现代特性使用偏少。 `auto` 和 `range-based for` 有一定使用，但 `structured binding` (C++17) 和 `lambda` 等高级特性几乎没有。

建议 (可选，不影响竞赛):

```
```cpp
// structured binding - 简化 pair 解包
auto [u, v] = edges[i]; // 代替 int u = edges[i].first;

// lambda + sort - 更灵活的排序
sort(a.begin(), a.end(), { return x.w < y.w; });
```
```

5.11 编译错误率

| 指标 | 数值 | 评价 |
|-------|-----|----------|
| CE 次数 | 247 | 10%，偏高 |
| 目标 | <5% | 竞赛选手合理水平 |

CE 率 10% 说明可能没有在本地编译就直接提交。**强烈建议**在本地配置 g++ 编译环境，提交前先编译通过。

5.12 代码风格综合评价

| 维度 | 评分 | 说明 |
|-----------|-------|-----------------------------------|
| 代码精简度 | ★★★★★ | 中位数 42 行，超长行仅 10 行 |
| IO 安全性 | ★★☆☆☆ | cin/cout 为主但 sync_with_stdio 仅 2% |
| 命名规范 | ★★★☆☆ | 竞赛标准，偏单字母，可读性一般 |
| 注释习惯 | ★★☆☆☆ | 0.66% 极低，复杂代码缺乏注释 |
| STL 熟练度 | ★★★☆☆ | 基础容器熟练，高级容器不足 |
| 模板规范度 | ★★☆☆☆ | 无固定模板头，缩进不统一 |
| 现代 C++ 运用 | ★★☆☆☆ | auto/range-for 偶尔用，其余少 |
| 算法模板多样性 | ★★★★☆ | BFS/DP/最短路/并查集丰富 |

5.13 代码风格改进建议 (优先级排序)

- 1. 紧急: 加 `ios::sync_with_stdio(false)` — 每份代码必加, 防止大数据 TLE
- 2. 紧急: Dijkstra 替代 SPFA — SPFA 容易被卡, Dijkstra+堆优化是标准
- 3. 重要: 统一缩进风格 — 选 Tab 或 4 空格, 不要混合
- 4. 重要: 降低 CE 率 — 本地编译后再提交, 目标 <5%
- 5. 重要: 固定模板头 — 建立个人竞赛模板, 包含常用宏和 IO 优化
- 6. 建议: 减少 `#define int long long` — 改用 `typedef long long ll`
- 7. 建议: 复杂题加注释 — 100+ 行代码至少 3-5 行关键注释
- 8. 建议: 多用 `set / unordered_map` — 扩展 STL 工具箱

推荐固定模板头

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int,int> pii;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fLL;

void solve() {
    // 写在这里
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;
    // cin >> T; // 多组数据时取消注释
    while (T--) solve();
    return 0;
}
```

六、六维能力评分

| 维度 | 评分 | 依据 |
|------|----|---------------------------------------|
| 基础算法 | 85 | 枚举/模拟/贪心/递归全面精通, 区间DP/搜索优化不足 |
| 数据结构 | 62 | 并查集22+线段树12+树状数组7; 平衡树/分块/莫队空白 |
| 图论 | 68 | 最短路31+拓扑17+二分图6+网络流5; Tarjan/树链剖分空白 |
| 动态规划 | 75 | CF 74题DP专练; 背包41+状压7+树形2; 区间DP/斜率优化不足 |
| 字符串 | 45 | KMP17+Manacher1; AC自动机/后缀数组/SAM 全空白 |
| 数学 | 40 | GCD50+筛法22; 快速幂/CRT/容斥/欧拉函数 全空白 |

八、训练路线图 (6 个月重点版)

第一阶段: 巩固 CSP-S 基础 (Month 1-2)

核心目标: 将 CSP-S 大纲中的空白项补齐, ABC 赛中稳定做到 D

Week 1-2: 数学专项

| 知识点 | 推荐题目 | 数量 |
|--------|----------------------|----|
| 快速幂 | P1226 快速幂 | 1 |
| 逆元 | P3811 乘法逆元 | 1 |
| 欧拉函数 | P2158 仪仗队, P2568 GCD | 2 |
| 扩展欧几里得 | P1082 同余方程 | 1 |
| CRT | P1495 中国剩余定理 | 1 |
| 容斥原理 | P2714 四元组统计 | 1 |

Week 3-4: 搜索优化专项

| 知识点 | 推荐题目 | 数量 |
|----------|-----------------------|----|
| 启发式搜索 A* | P1379 八数码, P2324 骑士精神 | 2 |
| 双向BFS | P1032 字串变换 | 1 |
| 迭代加深 | P1763 埃及分数 | 1 |

第二阶段: 数据结构突破 (Month 3-4)

核心目标: 掌握平衡树、分块、莫队等省选级数据结构

| 知识点 | 推荐题目 | 数量 |
|-------------|--------------------------|----|
| 平衡树 (Treap) | P3369 普通平衡树 | 1 |
| 分块 | P2801 教主的魔法, P3203 弹飞绵羊 | 2 |
| 莫队 | P1494 小Z的袜子, P2709 小B的询问 | 2 |
| 树链剖分 | P3384 树链剖分 | 1 |
| AC 自动机 | P3808 AC自动机(简单版) | 1 |

第三阶段: ABC 提速训练 (Month 5-6)

核心目标: ABC 赛中稳定做到 E

| 训练内容 | 频次 | 目的 |
|-----------------|---------------|---------|
| ABC D 题限时训练 | 5道/周 (限时15分钟) | 提速 D |
| ABC E 题训练 | 3道/周 (限时30分钟) | 稳定出 E |
| CF DP 1600-2000 | 3道/周 | 继续深化 DP |

每周训练时间分配建议

| 平台 | 内容 | 时间 | 目的 |
|------------|--------------|--------|-------|
| AtCoder | ABC 现场赛 | 100m/周 | 实战 |
| AtCoder | D/E 题限时训练 | 120m/周 | 提速 |
| Codeforces | DP 1600-2000 | 90m/周 | 深化 DP |
| Luogu | CSP-S 空白项补齐 | 自由 | 补短板 |

九、核心建议

- 1. 降低 CE 率: 本地配置编译环境, 提交前先编译。目标 CE 率 <5%
- 2. 培养分析式调试: WA 后先花 2 分钟分析原因再提交, 减少盲猜式调试
- 3. 补齐数学短板: 快速幂、欧拉函数、CRT 是 CSP-S 必考项
- 4. 适度冒险: 尝试做一些超出当前能力的省选级题目, 不要怕 WA
- 5. 建立模板库: 线段树/树剖/Tarjan 等模板整理成文件, 比赛时粘贴减少出错
- 6. 继续 DP 特训: CF 上的 DP 训练方向完全正确, 保持节奏

报告生成: 2026-05-28 | 数据源: Luogu + AtCoder + Codeforces 全平台 | 覆盖 888 份源码 + 2881 条提交记录